

调度部分的证明题

同济大学计算机系《操作系统》附加题作业 2025/12/04

授课教师 邓蓉

2025 拔尖/国豪班 姓名 毛星博 学号 2351520

1、IO bound 进程的优先级高于 CPU bound 进程，有利于缩短系统的平均周转时间。

证明：

假设系统中有两个并发进程 PA 和 PB：

PA：IO bound 进程（短作业），服务时间为 t 。

PB：CPU bound 进程（长作业），服务时间为 T 。

根据定义，IO bound 进程每次使用 CPU 时间极短，因此假设 $t \ll T$ 。

情况一：IO bound 优先级高

执行序列为：先执行 PA，再执行 PB。

PA 的周转时间 $TA = t$ （因为它不需要等待）。

PB 的周转时间 $TB = T + t$ （等待 PA 执行完）。

系统的平均周转时间 $T_1 = \frac{t + (t + T)}{2} = t + \frac{T}{2}$ 。

情况二：CPU bound 优先级高

执行序列为：先执行 PB，再执行 PA。

PB 的周转时间 $TB = T$ （它不需要等待）。

PA 的周转时间 $TA = T + t$ （等待 PB 执行完）。

系统的平均周转时间 $T_1 = \frac{T + (T + t)}{2} = T + \frac{t}{2}$ 。

计算两者之差： $\Delta T = T + \frac{t}{2} - \left(t + \frac{T}{2}\right) = \frac{T - t}{2}$

由于 $t \ll T$ （长作业时间远大于短作业），因此 $\Delta T > 0$

结论：让执行时间短的 IO bound 进程优先（情况一），系统的平均周转时间更短。

证毕。

2、在被调度集合固定的情况下，短作业优先算法理论最优，会给出平均周转时间最低的解。

证明：

反证法： 假设存在一个非 SJF 的调度序列 S 是最优的（即平均周转时间最小）

既然 S 不是 SJF 序列，那么在 S 中一定存在至少一对相邻的作业 P_i 和 P_j （ P_i 在 P_j 之前执行），服务时间满足 $T_i > T_j$ （即前面的作业比后面的长，这就叫“逆序”）。

我们交换 S 中 P_i 和 P_j 的位置，得到一个新的序列 S' 。

设交换前 P_i 开始执行的时间为 T_{Start} 。

计算交换前后的贡献变化：

在 S 中 P_i 的完成时间 $C_i = T_{Start} + T_i$ ； P_j 完成时间 $C_j = T_{Start} + T_i + T_j$

这两个作业对总周转时间的贡献 $Sum = C_i + C_j = 2T_{Start} + 2T_i + T_j$

在 S' 中 P_j 的完成时间 $C_i = T_{Start} + T_j$ ； P_i 完成时间 $C_j = T_{Start} + T_i + T_j$

这两个作业对总周转时间的贡献 $Sum' = C_i + C_j = 2T_{Start} + 2T_j + T_i$

$$Sum' - Sum = T_j - T_i$$

由于我们要消除的逆序满足 $T_i > T_j$ ，所以 $Sum' - Sum = T_j - T_i < 0$

所以，新序列 S' 的总周转时间（以及平均周转时间）比假设的“最优”序列 S 更小。这与 S 是最优序列的假设矛盾。

通过不断交换所有逆序对，我们最终会得到 SJF 序列，且每次交换都会减少周转时间。因此，SJF 算法给出的序列具有最短的平均周转时间。

证毕。

3、在时间片轮转的系统中，时间片越长，CPU bound 进程周转时间越短。

证明：

设 T_s : CPU bound 进程需要的总服务时间； t : 时间片长度（假设 T_s 是 t 的整数倍）；

n : 系统中与之并发的其他进程数量（背景负载）。

该进程需要执行的轮次数 $k = \frac{T_s}{t}$ 。

在每一轮中，系统会轮流调度所有 $n + 1$ 个进程。假设一轮的周期为 T_{cycle} ，背景负载与目标进程独立，所以 T_{cycle} 可以看成是一个随机变量。令随机变量 T_{cycle} 均值为 T_0 。

进程的周转时间 $TT = (\text{前 } k - 1 \text{ 轮的等待和执行时间}) + (\text{最后一轮的执行时间})$ 。

在前 $k - 1$ 轮中，进程每次都要经历完整的周期 T_{cycle} 。在最后一轮，进程执行完自己的时间片 t 后立刻完成，不需要再等待其他进程。

$$TT = (k - 1)T_{cycle} + t = \left(\frac{T_s}{t} - 1\right)T_0 + t$$

T_s 和 T_0 为常数，显然，随着 t 增大，总周转时间 TT 减小。

形象理解一下：时间片越长，总轮数越少，进程需要排队等待其他进程的次数就越少。

所以时间片越长，CPU bound 进程周转时间越短。

证毕。

参考答案

首先，我们要明确 IO bound 进程和 CPU bound 进程的运行模式。

CPU bound 进程，只运算、不 IO。比如，矩阵乘法程序。

IO bound 进程，只 IO，不运算。比如，示例 copy 程序。IO bound 进程每次使用 CPU，耗时极短，以该程序为例：

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

IO 完成后，copy 进程会被唤醒。唤醒后的 copy 进程，被调度上台后，使用 CPU：

- 执行 read 系统调用的下半段，将读入内存的 oldFile 文件数据（一个字节），送入用户空间，局部变量 c；随后，执行 write 系统调用的上半段，将 c 变量写入 newFile 文件，这个操作会写磁盘，所以，向磁盘发写命令后，copy 进程会睡，放弃 CPU。
- 或者。先执行 write 系统调用的下半段。。。再执行 read 系统调用的上半段。。。入睡放弃 CPU。

总之，每次唤醒后，执行时间很短，IO bound 进程就又睡了。

所以，考虑 CPU 调度，IO bound 进程的 Ts 很短，CPU bound 进程的 Ts 很长。

4、IO bound 进程的优先级高于 CPU bound 进程，有利于缩短系统的平均周转时间。

证明：假设系统并发 2 个进程，PA 是 IO bound 进程，服务时间 t；PB 是 CPU bound 进程，服务时间 T。 $t \ll T$ 。

- 若，IO bound 进程的优先级高于 CPU bound 进程。任务执行序列是 PA、PB。PA 周转时间是 t，PB 周转时间是 t+T。平均周转时间是 $T/2 + t$ 。
- 若，IO bound 进程的优先级小于 CPU bound 进程。任务执行序列是 PB、PA。PB 周转时间是 T，PA 周转时间是 t+T。平均周转时间是 $t/2 + T$ 。

$$(t/2 + T) - (T/2 + t) = (T-t)/2 > 0 \quad \text{得证。}$$

貌似差不多了。。。欢迎同学们补充。

5、作业。在被调度集合固定的情况下，短作业优先算法理论最优，会给出平均周转时间最低的解。

证明：

3、在时间片轮转的系统中，时间片越长，CPU bound 进程周转时间越短。我们做的题，答

案好像是这么回事儿。试着证明。

背景负载 \ 时间片	10ms	50ms	100ms
4	460ms	300ms	100ms

证明：令时间片为 t 。CPU bound 任务服务时间为 T_s 。不失一般性，令 T_s 是 t 的整数倍。
这个任务需要经历 $\lceil T_s/t \rceil$ 轮才能运行结束。每一轮 T 秒，目标进程可以执行一个时间片 t 。

t 是调度系统的配置参数，它是一个常数。 T 是一个随机变量，具体的值与背景负载（与进程并发的其它进程）的进程数量和每个进程的具体行为相关。背景负载与目标进程独立，所以 T 可以看成是一个随机变量。令随机变量 T 均值为 T_0 ，CPU bound 进程的周转时间为 $(\lceil T_s/t \rceil - 1) * T_0 + t$ ，其中 $T_0 > t$ ；

所以，时间片 t 越长，周转时间越小。
貌似差不多了。。。欢迎同学们补充。